DiffTracker VersionControl Documentation

April 23, 2026

Contents

1 Conceptual Definitions

Definition 1.1 (Multi-Agent System)**.** A multi-agent system consists of independent agents editing the same file. Each agent records its changes without knowledge of other agents.

Definition 1.2 (Agent)**.** An agent is an entity that edits a file. Each agent has a unique positive integer ID.

1.1 The Base File

Definition 1.3 (Base File)**.** The base file is the original file state before edits. It is a list of text lines. Example: `["name = Alice", "greeting = hello", "print greeting"]`.

Definition 1.4 (Line Number)**.** Line numbers are 1-indexed and half-open: range `[start, stop)` includes `start` and excludes `stop`. Example: `[2, 4]` covers lines 2 and 3.

1.2 Edits and Hunks

Definition 1.5 (Edit)**.** An edit is one of:

- Insert: add lines at a position.
- Delete: remove lines.
- Replace: remove lines and add new ones.

Definition 1.6 (Operation)**.** An operation (`op`) is one of `insert`, `delete`, `replace`.

Definition 1.7 (Range)**.** A range `[start, stop)` identifies lines in the base file.

- `start` included, `stop` excluded.
- Width = `stop - start`.
- For insert: `start = stop`.

Line numbers are 1-based.

Definition 1.8 (Before Lines)**.** `before` holds the exact lines from the base file at `range`. For insert it is empty.

Definition 1.9 (After Lines)**.** `after` holds the lines to insert. For delete it is empty.

Definition 1.10 (Hunk)**.** A hunk is one atomic edit. It contains `seq`, `op`, `range`, `before`, `after`. See Definitions 1.7, 1.6, 1.8, 1.9.

1.3 Metadata

Definition 1.11 (Agent ID)**.** Agent ID is a unique positive integer ≥ 1. It appears at top level of every diff.

Definition 1.12 (Timestamp)**.** Timestamp is an ISO 8601 date-time string.

Definition 1.13 (File Path)**.** File path is a non-empty string identifying the target file.

1.4 Diff

Definition 1.14 (Base Line Count)**.** Base line count is the number of lines in the base file (≥ 0).

Definition 1.15 (Diff)**.** A diff contains agent ID, file path, timestamp, base line count, and ordered list of hunks.

1.5 Constraints

Definition 1.16 (Non-Overlap)**.** No two hunks in one diff may have overlapping ranges: `a < d \land c < b` for ranges `[a,b)`, `[c,d)`.

Definition 1.17 (Schema Validity)**.** A diff satisfies the JSON schema if all required fields, types, bounds, shapes, and non-overlap hold.

Definition 1.18 (Base Alignment)**.** A hunk is base-aligned when `before` matches the base file slice at its range. A diff is base-aligned when all hunks are aligned and `base_line_count` matches base length.

2 Reading Order

Read the files in this order.

1. `Basic.lean`
2. `Defns.lean`
3. `JsonParse.lean`
4. `Semantics.lean`
5. `Conflict.lean`
6. `History.lean`
7. `FullMerge.lean`

Dependency notes:

- `Defns.lean` needs `Basic.lean`.
- `JsonParse.lean`, `Semantics.lean`, and `Conflict.lean` need `Defns.lean`.
- `FullMerge.lean` needs `Semantics.lean`, `Conflict.lean`, and `History.lean`.

Section 11 lists work that is not part of this version.

3 JSON Schema

Source: `schemas/difftracker.schema.json`.

Lean mirror: `Defns.lean` (structured diff types), `JsonParse.lean` (parser). Lean declarations use the `VersionControl` namespace.

The schema defines the wire format for one `Diff` (Definition 1.15). Commit ancestry is stored separately.

All terms are defined in Section 1. This section maps schema keywords to constraints.

3.1 Top-Level Object

The top-level object must contain exactly these fields (`"required"` and `"additionalProperties": false`):

- `agent`: `integer`, `minimum: 1`.
- `file`: `string`, `minLength: 1`.
- `timestamp`: `string` with `format: date-time`.
- `base_line_count`: `integer`, `minimum: 0`.
- `diffs`: `array` of objects matching `"\#/\$defs/hunk"`.

3.2 Hunk Object

Each element of `diffs` is a hunk (`"\$ref": "\#/\$defs/hunk"`). It must contain exactly these fields (`"required"` and `"additionalProperties": false`):

- `seq`: `integer`, `minimum: 1`.
- `op`: one of `"insert"`, `"delete"`, `"replace"` (`"enum"`).
- `range`: array of exactly two integers ≥ 1 (`minItems: 2`, `maxItems: 2`, `items: \{type: integer, minimum: 1\}`).
- `before`: array of strings.
- `after`: array of strings.

The `allOf` array with `if`/`then` clauses enforces operation shapes:

- `insert`: `before` empty, `after` non-empty.
- `delete`: `before` non-empty, `after` empty.
- `replace`: both `before` and `after` non-empty.

3.3 Range Semantics

`range` is a half-open interval `[start, stop)` with 1-based indexing.

- `start` is included, `stop` is excluded.
- Width = `stop - start`.
- `start = stop` denotes an insert position.
- Valid for base file of `N` lines: `1 <= start <= stop <= N+1`.

Examples:

- `[2,4]` affects lines 2 and 3.
- `[1,1]` inserts before line 1.
- `[N+1,N+1]` appends to end of file.

3.4 Lean Constraints

The schema enforces structure. Lean enforces additional invariants in `Diff.checkSchema` and `Hunk.checkSchema`:

- `agent >= 1` and `file != ""`.
- No two hunks overlap within one diff, and no two inserts share the same position within one diff.
- `stop <= base_line_count + 1`.
- For `delete` and `replace`: `before.length = stop - start`.
- `start <= stop`.

3.5 Schema vs Lean Boundary

- Schema: rejects unknown fields (`additionalProperties: false`), enforces types, `enum`, array lengths, basic shapes via `allOf/if/then`.
- Lean: stores `timestamp` as `String`, requires the five top-level fields when parsing, and validates inter-field relations and non-overlap.

JSON Schema defines the wire format. Lean defines semantic validity.

3.6 Base Alignment

`Diff.checkAgainstBase` and `Hunk.baseAligned` verify:

- `before` exactly matches `slice(base, range)`.
- `base_line_count = base.length`.

These checks occur before `applyDiff`.

3.7 Example

Base file:

```
name = Alice
greeting = hello
print greeting
```

Diff:

```
{
  "agent": 1,
  "file": "hello.py",
  "timestamp": "2026-04-21T10:00:00Z",
  "base_line_count": 3,
  "diffs": [
    {
      "seq": 1,
      "op": "insert",
      "range": [1, 1],
      "before": [],
      "after": ["# my script"]
    },
    {
      "seq": 2,
      "op": "replace",
      "range": [2, 3],
      "before": ["greeting = hello"],
      "after": ["greeting = hello world"]
    }
  ]
}
```

Result after apply:

```
# my script
name = Alice
greeting = hello world
print greeting
```

4 Basic.lean

`Basic.lean` is the repository foundation used by the V1 structured diff layer. The structured diff modules import this foundation rather than replacing it.

4.1 Imports

Definition 4.1 (No imports)**.** `[Basic.lean]` has no imports. It uses Lean core declarations only.

4.2 Repository Primitives

Definition 4.2 (File)**.** `[File]` is an abbreviation for `String`. It denotes a file path or file identifier.

Definition 4.3 (Content)**.** `[Content]` is an abbreviation for `String`. It denotes the complete contents of a file as one string.

Definition 4.4 (Agent)**.** `[Agent]` is an abbreviation for `Nat`. It denotes an agent id.

Definition 4.5 (Repository)**.** `[Repository]` is an abbreviation for `File -> Option Content`. It is a partial map from file names to contents.

Definition 4.6 (System)**.** `[System]` is a structure with fields `origin : Repository` and `agents : Agent -> Repository`. It represents one central repository and one local repository for each agent.

Definition 4.7 (Timeline)**.** `[Timeline]` is an inductive type indexed by `System`. It has two constructors: `origin` and `agentic agent`.

4.3 Whole-File Patch Skeleton

Definition 4.8 (FilePatch)**.** `[FilePatch]` is an abbreviation for `File -> Option Content`. It is the existing whole-file patch concept from the repository foundation.

Definition 4.9 (Commit)**.** `[Commit]` is a structure with fields `author : Agent`, `changes : FilePatch`, and `commitMessage : String`.

4.4 Existing Action Skeleton

`Basic.lean` also contains an `ActionF/ActionM` skeleton for repository-level actions. The structured diff layer leaves that skeleton in place. It only changes the old whole-file patch name from `Diff` to `FilePatch`, because the structured JSON diff introduced in `Defns.lean` is named `Diff`.

Definition 4.10 (Compatibility rename)**.** The compatibility edit is:

$$\texttt{Diff := File -> Option Content} \quad \longrightarrow \quad \texttt{FilePatch := File -> Option Content}.$$

The `Commit.changes` field is updated accordingly.

4.5 Relationship To The Structured Diff Stack

Definition 4.11 (Separation of roles)**.** `Basic.lean` does not define structured line-edit declarations such as `Range`, `Hunk`, or the JSON-facing `Diff`. Those belong to `Defns.lean`. The older action skeleton in `Basic.lean` is preserved as existing repository code and remains separate from the structured diff core.

5 Defns.lean

`Defns.lean` defines the structured diff layer. This file matches the JSON schema: a file is represented as lines, one edit is represented as a hunk, and one agent's edits to one file are represented as a structured `Diff`.

5.1 Imports

Definition 5.1 (import Mathlib.Tactic)**.** `[import Mathlib.Tactic]` imports proof tactics used in this file, including `aesop` and `omega`.

Definition 5.2 (import Lean.Data.Json.Basic)**.** `[import Lean.Data.Json.Basic]` imports Lean's `Json` datatype.

Definition 5.3 (Lean JSON FromToJson import)**.** `[import Lean.Data.Json.FromToJson.Basic]` imports `FromJson`, `ToJson`, `fromJson?`, and `toJson`.

Definition 5.4 (import Init.Data.List.Sort.Basic)**.** `[import Init.Data.List.Sort.Basic]` imports `List.mergeSort`, used by `Diff.sortedHunks`.

Definition 5.5 (import VersionControl.Basic)**.** `[import VersionControl.Basic]` imports the repository-level primitives `Agent` and `File`.

Definition 5.6 (open Lean)**.** `[open Lean]` makes `Json`, `fromJson?`, and `toJson` available without the `Lean.` prefix.

5.2 File Lines and Operations

Definition 5.7 (BaseFile)**.** `[BaseFile]` is an abbreviation for `List String`. It represents a file as a list of lines.

Definition 5.8 (Op)**.** `[Op]` is an inductive type with constructors `insert`, `delete`, and `replace`.

Definition 5.9 (deriving)**.** `[deriving]` asks Lean to generate standard instances. In this file, `Repr` supports printing, `DecidableEq` supports decidable equality, `Inhabited` gives a default value, `BEq` gives boolean equality, `FromJson` parses JSON, and `ToJson` serializes JSON.

5.3 Range

Definition 5.10 (Range)**.** `[Range]` is a structure with fields `start : Nat` and `stop : Nat`. It represents the half-open, 1-indexed interval `[start, stop)` in the original file.

Definition 5.11 (Range.width)**.** `[Range.width]` returns `r.stop - r.start`.

Definition 5.12 (Range.isInsert)**.** `[Range.isInsert]` states `r.start = r.stop`. This marks an insert point.

Definition 5.13 (Range.wellFormed)**.** `[Range.wellFormed]` states that `1 <= r.start`, `r.start <= r.stop`, and `r.stop <= baseLineCount + 1`.

Definition 5.14 (Range.overlaps)**.** `[Range.overlaps]` states that two half-open ranges share at least one targeted original line.

Definition 5.15 (Range.pointInsertCollision)**.** `[Range.pointInsertCollision]` states that two ranges are both insert points at the same line.

Theorem 5.1 (Range.overlaps_comm)**.** `[Range.overlaps_comm]` states `left.overlaps right <-> right.overlaps left`.

Theorem 5.2 (Range.width_eq_zero_of_isInsert)**.** `[Range.width_eq_zero_of_isInsert]` states that an insert range has width zero.

Theorem 5.3 (Range.isInsert_iff_width_eq_zero)**.** `[Range.isInsert_iff_width_eq_zero]` states that, when `r.start <= r.stop`, `r.isInsert` is equivalent to `r.width = 0`.

Definition 5.16 (Decidable Range.wellFormed)**.** `[Decidable Range.wellFormed]` lets Lean compute whether a range is valid for a declared base line count.

Definition 5.17 (FromJson Range)**.** `[FromJson Range]` parses a range from a two-element JSON array `[start, stop]`.

Definition 5.18 (ToJson Range)**.** `[ToJson Range]` serializes a range as `[start, stop]`.

5.4 Slicing

Definition 5.19 (slice)**.** `[slice]` returns the lines of a `BaseFile` covered by a `Range`. It drops `range.start - 1` lines and takes `range.width` lines.

Theorem 5.4 (slice_of_insert)**.** `[slice_of_insert]` states that slicing an insert range returns `[]`.

5.5 Hunk

Definition 5.20 (Hunk)**.** `[Hunk]` is a structure with fields `seq`, `op`, `range`, `before`, and `after`. It represents one edit operation.

Definition 5.21 (Hunk.opMatchesShape)**.** `[Hunk.opMatchesShape]` states the operation-specific shape rule. Insert requires empty `before` and nonempty `after`. Delete requires a positive-width range, matching `before.length`, and empty `after`. Replace requires a positive-width range, matching `before.length`, and nonempty `after`.

Definition 5.22 (Hunk.baseAligned)**.** `[Hunk.baseAligned]` states `h.before = slice base h.range`.

Definition 5.23 (Hunk.schemaWellFormed)**.** `[Hunk.schemaWellFormed]` checks hunk validity using only `baseLineCount`. It checks sequence number, range bounds, and operation shape.

Definition 5.24 (Hunk.wellFormed)**.** `[Hunk.wellFormed]` checks hunk validity against an actual `BaseFile`. It adds the exact `before`-line alignment check.

Definition 5.25 (Hunk.compareByRange)**.** `[Hunk.compareByRange]` orders hunks by range start and then range stop.

Definition 5.26 (Decidable Hunk.opMatchesShape)**.** `[Decidable Hunk.opMatchesShape]` lets Lean compute the hunk shape rule.

Definition 5.27 (Hunk.checkSchema)**.** `[Hunk.checkSchema]` executes hunk validation and returns `Except String Unit`.

Theorem 5.5 (Hunk.opMatchesShape_insert)**.** `[Hunk.opMatchesShape_insert]` states that an insert hunk with empty `before` and nonempty `after` satisfies `opMatchesShape` exactly when its range is an insert range.

5.6 Diff

Definition 5.28 (Diff)**.** `[Diff]` is a structure with fields `agent : Agent`, `file : File`, `timestamp : String`, `base_line_count : Nat`, and `diffs : List Hunk`. It represents one agent's edits to one file.

Definition 5.29 (FromJson Diff)**.** `[FromJson Diff]` parses the required JSON fields `agent`, `file`, `timestamp`, `base_line_count`, and `diffs`.

Definition 5.30 (ToJson Diff)**.** `[ToJson Diff]` serializes a `Diff` to a JSON object with the same five fields.

Definition 5.31 (Diff.metadataWellFormed)**.** `[Diff.metadataWellFormed]` states `diff.agent >= 1` and `diff.file <> ""`.

Definition 5.32 (Diff.sortedHunks)**.** `[Diff.sortedHunks]` returns `diff.diffs` sorted by `Hunk.compareByRange`.

Definition 5.33 (Diff.pairwiseSeparated)**.** `[Diff.pairwiseSeparated]` states that no two hunks overlap and no two insert hunks collide at the same insertion point.

Definition 5.34 (Decidable Diff.pairwiseSeparated)**.** `[Decidable Diff.pairwiseSeparated]` supplies the decision procedures used to compute hunk separation.

Definition 5.35 (Diff.schemaWellFormed)**.** `[Diff.schemaWellFormed]` states full schema validity without an actual base file. It checks metadata, sorted hunk separation, and `Hunk.schemaWellFormed` for every hunk.

Definition 5.36 (Diff.wellFormed)**.** `[Diff.wellFormed]` states full validity against an actual `BaseFile`. It checks metadata, base line count equality, hunk separation, and `Hunk.wellFormed` for every hunk.

Definition 5.37 (Diff.checkSchema)**.** `[Diff.checkSchema]` executes diff schema validation and returns `Except String Unit`.

Theorem 5.6 (Diff.sortedHunks_nil)**.** `[Diff.sortedHunks_nil]` states that sorting the empty hunk list of a concrete empty diff returns the empty list.

5.7 Downstream Use

`JsonParse.lean` uses `FromJson Diff` and `ToJson Diff`. `Semantics.lean` uses `BaseFile`, `slice`, `Hunk.baseAligned`, `Diff.checkSchema`, and `Diff.sortedHunks`. `Conflict.lean` uses `Range`, `Hunk`, and `Diff`. `History.lean` and `FullMerge.lean` use `Diff` as the payload for history and merge computations.

6 JsonParse.lean

`JsonParse.lean` defines JSON entry points for the structured `Diff` type. The file does not define the JSON schema itself. The schema shape is implemented by the `FromJson Diff` and `ToJson Diff` instances from `Defns.lean`. This file gives those instances short, named wrappers.

6.1 Overall Structure

The file has three layers.

1. Imports and namespace setup.
2. Three executable functions: `parseDiffJson`, `parseDiff`, and `diffAsJson`.
3. Two definitional theorems describing exactly what the wrappers expand to.

6.2 Imports and Namespace

Definition 6.1 (import VersionControl.Defns)**.** `[import VersionControl.Defns]` imports `Diff`, `FromJson Diff`, `ToJson Diff`, and the `VersionControl` namespace declarations needed by this file.

Definition 6.2 (open Lean)**.** `[open Lean]` makes `Json`, `Json.parse`, `fromJson?`, and `toJson` available without writing the `Lean.` prefix.

Definition 6.3 (namespace VersionControl)**.** `[namespace VersionControl]` places the parsing functions and theorems in the same namespace as `Diff`. The full names are `VersionControl.parseDiffJson`, `VersionControl.parseDiff`, and `VersionControl.diffAsJson`.

6.3 JSON and Except

Definition 6.4 (Json)**.** `[Json]` is Lean's datatype for already-parsed JSON values. A value of type `Json` is not raw text. It is a structured JSON tree.

Definition 6.5 (Json.parse)**.** `[Json.parse]` has type `String -> Except String Json`. It reads raw JSON text. If the text is syntactically invalid JSON, it returns `Except.error message`. If the text is syntactically valid JSON, it returns `Except.ok json`.

Definition 6.6 (Except String alpha)**.** `[Except String alpha]` is a result type with two cases: `Except.ok value` for success and `Except.error message` for failure. In this file, parsing functions use `Except String` so syntax errors and schema-conversion errors can carry an error message.

Definition 6.7 (do notation)**.** `[do]` notation sequences computations that can fail. In `parseDiff`, the line `let json <- Json.parse text` means: parse the raw text; if parsing fails, return that error immediately; if parsing succeeds, bind the parsed JSON value to `json` and continue.

6.4 Functions

Definition 6.8 (parseDiffJson)**.** `[parseDiffJson]` has type `Json -> Except String Diff`. It converts an already-parsed JSON value into a `Diff`. It is exactly `fromJson? json` specialized to `Diff`.

Definition 6.9 (fromJson?)**.** `[fromJson?]` is the generic Lean JSON conversion function. In this file, `fromJson? json` uses the `FromJson Diff` instance defined in `Defns.lean`, because the expected result type is `Diff`.

Definition 6.10 (parseDiff)**.** `[parseDiff]` has type `String -> Except String Diff`. It is the main entry point for raw JSON text. It first calls `Json.parse`; then it calls `parseDiffJson` on the parsed JSON value.

Definition 6.11 (diffAsJson)**.** `[diffAsJson]` has type `Diff -> Json`. It serializes a `Diff` to a JSON value by calling `toJson diff`.

Definition 6.12 (toJson)**.** `[toJson]` is the generic Lean JSON serialization function. In this file, `toJson diff` uses the `ToJson Diff` instance defined in `Defns.lean`, because the input value has type `Diff`.

6.5 Parsing Pipeline

The raw-text parsing pipeline is:

1. Input: `text : String`.
2. `Json.parse text` checks JSON syntax.
3. If JSON syntax fails, `parseDiff text` returns the syntax error.
4. If JSON syntax succeeds, Lean has `json : Json`.
5. `parseDiffJson json` checks whether the JSON has the fields and types required by `Diff`.
6. If conversion fails, `parseDiff text` returns the conversion error.
7. If conversion succeeds, `parseDiff text` returns `Except.ok diff`.

This file does not call `Diff.checkSchema`. Therefore `parseDiff text = Except.ok diff` means the JSON was converted into a `Diff` value. To check hunk shape and pairwise non-overlap, call `diff.checkSchema` after parsing.

6.6 Theorems

Theorem 6.1 (parseDiff_eq_parseDiffJson)**.** `[parseDiff_eq_parseDiffJson]` states that `parseDiff text` is exactly the two-step computation that first runs `Json.parse text` and then runs `parseDiffJson json`.

Theorem 6.2 (parseDiffJson_eq_fromJson)**.** `[parseDiffJson_eq_fromJson]` states that `parseDiffJson json` is definitionally equal to `fromJson? json` specialized to `Except String Diff`.

6.7 Downstream Use

`JsonParse.lean` feeds into `Semantics.lean`. After JSON has been parsed into a `Diff`, the next operation is applying that `Diff` to a `BaseFile`, which is described in `Semantics.lean`.

6.8 Integration Note

The repository build checks this file through `VersionControl.lean`.

7 Semantics.lean

`Semantics.lean` defines how a structured `Diff` is applied to a `BaseFile`. It also defines inversion functions for producing the syntactic undo of an operation, hunk, or diff.

7.1 Imports and Namespace

This module depends on `Defns.lean` for the structured diff types and helper functions, and all declarations remain in the `VersionControl` namespace.

7.2 Base Checking

Definition 7.1 (namespace Diff)**.** `[namespace Diff]` places `checkAgainstBase` inside the `Diff` namespace. Its full name is `VersionControl.Diff.checkAgainstBase`.

Definition 7.2 (Diff.checkAgainstBase)**.** `[Diff.checkAgainstBase]` has type `Diff -> BaseFile -> Except String Unit`. It first runs `diff.checkSchema`. Then it checks that `diff.base_line_count = base.length`. It also checks that each hunk's `before` field equals `slice base hunk.range`. If all checks pass, it returns `Except.ok ()`. Otherwise it returns an error message.

7.3 Unchanged Lines

Definition 7.3 (unchangedSegment)**.** `[unchangedSegment]` has type `BaseFile -> Nat -> Nat -> List String`. It returns the unchanged lines from `cursor` inclusive to `nextStart` exclusive. It computes this by dropping `cursor - 1` lines and taking `nextStart - cursor` lines.

7.4 Applying Hunks

Definition 7.4 (applyOrderedHunks)**.** `[applyOrderedHunks]` has type `BaseFile -> Nat -> List Hunk -> Except String BaseFile`. It applies a list of hunks to a base file, assuming the hunks are already ordered from left to right by range.

Definition 7.5 (applyOrderedHunks empty case)**.** The empty-list case of `[applyOrderedHunks]` returns `base.drop (cursor - 1)`. This copies the remaining suffix of the original file after all hunks have been processed.

Definition 7.6 (applyOrderedHunks hunk case)**.** The nonempty-list case of `[applyOrderedHunks]` processes `hunk :: rest`. It checks four conditions before producing output: `hunk.range.wellFormed base.length`, `hunk.range.start >= cursor`, `hunk.before = slice base hunk.range`, and `hunk.opMatchesShape`. If any check fails, it returns `Except.error`. If all checks pass, it recursively applies the remaining hunks from cursor `hunk.range.stop` and returns `unchangedSegment base cursor hunk.range.start ++ hunk.after ++ tail`.

Definition 7.7 (applyHunk)**.** `[applyHunk]` has type `Hunk -> BaseFile -> Except String BaseFile`. It applies one hunk by calling `applyOrderedHunks base 1 [hunk]`.

Definition 7.8 (applyDiff)**.** `[applyDiff]` has type `Diff -> BaseFile -> Except String BaseFile`. It first calls `diff.checkAgainstBase base`. Then it calls `applyOrderedHunks base 1 diff.sortedHunks`. Therefore schema validation, base line-count validation, hunk sorting, hunk alignment, and hunk application are all part of the main diff-application path.

7.5 Apply Theorems

Theorem 7.1 (applyDiff_empty)**.** `[applyDiff_empty]` states that applying a valid empty diff to any base file returns `Except.ok base`. The theorem requires the file name to be nonempty.

Theorem 7.2 (applyHunk_insert_at_start)**.** `[applyHunk_insert_at_start]` states that applying an insert hunk with range `\{ start := 1, stop := 1 \}`, empty `before`, and `after := [line]` returns `Except.ok (line :: base)`.

7.6 Inversion

Definition 7.9 (invertOp)**.** `[invertOp]` has type `Op -> Op`. It maps `insert` to `delete`, maps `delete` to `insert`, and maps `replace` to `replace`.

Definition 7.10 (invertHunk)**.** `[invertHunk]` has type `Hunk -> Hunk`. It swaps the `before` and `after` lists and replaces `op` with `invertOp h.op`. The range and sequence number are unchanged.

Definition 7.11 (hunkLineDelta)**.** `[hunkLineDelta]` computes how many lines a hunk adds or removes: `after.length - before.length`.

Definition 7.12 (inverseRangeOnPatched)**.** `[inverseRangeOnPatched]` computes the inverse hunk range in the coordinate space of the patched file rather than the original base file.

Definition 7.13 (invertOrderedHunks)**.** `[invertOrderedHunks]` walks sorted hunks left to right, tracks the net line offset introduced by earlier hunks, and builds inverse hunks using patched coordinates.

Definition 7.14 (resultLineCount)**.** `[resultLineCount]` computes the patched-file line count that becomes the `base\_line\_count` of the inverse diff.

Definition 7.15 (invertDiff)**.** `[invertDiff]` has type `Diff -> Diff`. It builds inverse hunks in patched-file coordinates, updates `base\_line\_count` to the patched-file length, and stores the inverse hunks in sorted application order.

Theorem 7.3 (invertHunk_involution)**.** `[invertHunk_involution]` states that applying `invertHunk` twice returns the original hunk: `invertHunk (invertHunk h) = h`.

7.7 Execution Pipeline

The main execution path is:

1. `parseDiff` in `JsonParse.lean` converts JSON text into a `Diff`.
2. `Diff.checkSchema` in `Defns.lean` checks schema-level hunk validity.
3. `Diff.checkAgainstBase` in `Semantics.lean` checks that the supplied base file has the declared line count and that each hunk's `before` lines match the base file.
4. `applyDiff` sorts hunks with `Diff.sortedHunks`.
5. `applyOrderedHunks` validates hunk alignment against the actual base file and builds the output file.

8 Conflict.lean

`Conflict.lean` defines when two hunks or two diffs conflict. It gives both logical specifications in `Prop` and executable boolean checks in `Bool`. It also proves that the boolean checks match the logical specifications.

8.1 Imports and Namespace

This module depends on `Defns.lean` for `Range`, `Hunk`, and `Diff`, and uses `Mathlib.Tactic` for the short symmetry and reflection proofs. All declarations remain in the `VersionControl` namespace.

8.2 Logical Conflict Definitions

Definition 8.1 (identicalChange)**.** `[identicalChange]` has type `Hunk -> Hunk -> Prop`. It states that two hunks have the same `range`, the same `before` lines, and the same `after` lines. The `seq` and `op` fields are not part of this equality test.

Definition 8.2 (pointInsertConflict)**.** `[pointInsertConflict]` has type `Hunk -> Hunk -> Prop`. It states that both hunks are insert hunks, both insert at the same start line, and their `after` lists are different.

Definition 8.3 (hunksConflict)**.** `[hunksConflict]` has type `Hunk -> Hunk -> Prop`. It states that two hunks conflict when they are not identical and either their ranges overlap or they have a point-insert conflict.

Definition 8.4 (diffsConflict)**.** `[diffsConflict]` has type `Diff -> Diff -> Prop`. It states that two diffs conflict when there exists a hunk in the left diff and a hunk in the right diff such that the two hunks conflict.

Definition 8.5 (compatible)**.** `[compatible]` has type `Diff -> Diff -> Prop`. It states that every hunk in the left diff is non-conflicting with every hunk in the right diff.

8.3 Decidable Instances

Definition 8.6 (Decidable identicalChange)**.** `[Decidable identicalChange]` supplies a decision procedure for `identicalChange left right`.

Definition 8.7 (Decidable Range.overlaps)**.** `[Decidable Range.overlaps]` supplies a decision procedure for `left.overlaps right`.

Definition 8.8 (Decidable pointInsertConflict)**.** `[Decidable pointInsertConflict]` supplies a decision procedure for `pointInsertConflict left right`.

8.4 Boolean Conflict Checks

Definition 8.9 (hunkConflictFlag)**.** `[hunkConflictFlag]` has type `Hunk -> Hunk -> Bool`. It is the executable hunk-conflict check. It returns `true` exactly when the hunks are not identical and either their ranges overlap or they have a point-insert conflict.

Definition 8.10 (diffConflictFlag)**.** `[diffConflictFlag]` has type `Diff -> Diff -> Bool`. It is the executable diff-conflict check. It returns `true` exactly when some pair of hunks, one from each diff, has `hunkConflictFlag = true`.

Definition 8.11 (decide)**.** `[decide]` converts a decidable proposition into a boolean. This file uses `decide` to compute `identicalChange`, `Range.overlaps`, and `pointInsertConflict` inside `hunkConflictFlag`.

8.5 Hunk Theorems

Theorem 8.1 (identicalChange_comm)**.** `[identicalChange_comm]` states that `identicalChange` is symmetric: `identicalChange left right <-> identicalChange right left`.

Theorem 8.2 (not_hunksConflict_of_identicalChange)**.** `[not_hunksConflict_of_identicalChange]` states that identical hunks do not conflict.

Theorem 8.3 (hunksConflict_comm)**.** `[hunksConflict_comm]` states that hunk conflict is symmetric: `hunksConflict left right <-> hunksConflict right left`.

Theorem 8.4 (identical_hunks_not_conflict)**.** `[identical_hunks_not_conflict]` states that a hunk does not conflict with itself.

Theorem 8.5 (separated_hunks_not_conflict)**.** `[separated_hunks_not_conflict]` states that two hunks do not conflict if their ranges are separated and there is no point-insert conflict.

Theorem 8.6 (same_point_equal_insert_not_conflict)**.** `[same_point_equal_insert_not_conflict]` states that two inserts at the same point with the same inserted text do not conflict, assuming both `before` lists are empty.

8.6 Boolean Reflection Theorems

Theorem 8.7 (hunkConflictFlag_eq_true_iff)**.** `[hunkConflictFlag_eq_true_iff]` states that `hunkConflictFlag left right = true` is equivalent to `hunksConflict left right`.

Theorem 8.8 (hunkConflictFlag_eq_false_iff)**.** `[hunkConflictFlag_eq_false_iff]` states that `hunkConflictFlag left right = false` is equivalent to `not hunksConflict left right`.

Theorem 8.9 (identical_hunks_not_flagged)**.** `[identical_hunks_not_flagged]` states that `hunkConflictFlag hunk hunk = false`.

Theorem 8.10 (same_point_equal_insert_not_flagged)**.** `[same_point_equal_insert_not_flagged]` is the boolean version of `same_point_equal_insert_not_conflict`. It states that equal same-point insert hunks produce `hunkConflictFlag = false`.

8.7 Diff Theorems

Theorem 8.11 (diffConflictFlag_eq_true_iff)**.** `[diffConflictFlag_eq_true_iff]` states that `diffConflictFlag left right = true` is equivalent to `diffsConflict left right`.

Theorem 8.12 (diffConflictFlag_eq_false_iff)**.** `[diffConflictFlag_eq_false_iff]` states that `diffConflictFlag left right = false` is equivalent to `not diffsConflict left right`.

Theorem 8.13 (flagged_implies_real_conflict)**.** `[flagged_implies_real_conflict]` states that if the executable diff flag returns `true`, then the logical proposition `diffsConflict` holds.

Theorem 8.14 (real_conflict_implies_flagged)**.** `[real_conflict_implies_flagged]` states that if `diffsConflict` holds, then the executable diff flag returns `true`.

Theorem 8.15 (diffsConflict_comm)**.** `[diffsConflict_comm]` states that diff conflict is symmetric: `diffsConflict left right <-> diffsConflict right left`.

Theorem 8.16 (compatible_iff_not_diffsConflict)**.** `[compatible_iff_not_diffsConflict]` states that `compatible left right` is equivalent to `not diffsConflict left right`.

Theorem 8.17 (compatible_empty_left)**.** `[compatible_empty_left]` states that an empty diff on the left is compatible with any diff.

Theorem 8.18 (compatible_empty_right)**.** `[compatible_empty_right]` states that an empty diff on the right is compatible with any diff.

Theorem 8.19 (compatible_comm)**.** `[compatible_comm]` states that compatibility is symmetric: `compatible left right <-> compatible right left`.

8.8 Execution Pipeline

The conflict-checking pipeline is:

1. `hunkConflictFlag` computes conflict for two hunks.
2. `diffConflictFlag` computes whether any hunk pair conflicts.
3. `hunkConflictFlag_eq_true_iff` connects the hunk boolean check to `hunksConflict`.
4. `diffConflictFlag_eq_true_iff` connects the diff boolean check to `diffsConflict`.
5. `compatible_iff_not_diffsConflict` states compatibility as the absence of diff conflict.

9 History.lean

`History.lean` defines linear patch histories. A history is a list of steps, where each step has a base commit id, a new commit id, and a structured `Diff`. The file also defines shared-prefix splitting for two histories.

9.1 Imports and Namespace

This module depends on `Defns.lean` for `Diff` and `Diff.metadataWellFormed`, and it stays in the shared `VersionControl` namespace.

9.2 Commit Ids and Patch Steps

Definition 9.1 (CommitId)**.** `[CommitId]` is an abbreviation for `String`. A commit id is just a short string name such as `init`, `a1`, or `left1`.

Definition 9.2 (PatchStep)**.** `[PatchStep]` is a structure with fields `baseCommit`, `newCommit`, and `diff`. It represents one linear step from one commit id to the next.

Definition 9.3 (PushHistory)**.** `[PushHistory]` is an abbreviation for `List PatchStep`. It represents a linear sequence of patch steps.

Definition 9.4 (PatchStep.wellFormed)**.** `[PatchStep.wellFormed]` states that both commit ids are nonempty and that the diff metadata is well formed.

9.3 Commit Paths and Distinctness

Definition 9.5 (commitPath)**.** `[commitPath]` converts a history into the sequence of commit ids it visits. For `[init -> a1, a1 -> a2]`, the commit path is `[init, a1, a2]`.

Definition 9.6 (distinctCommitIds)**.** `[distinctCommitIds]` states that the commit path has no duplicate commit ids.

9.4 History Validity

Definition 9.7 (validHistory)**.** `[validHistory]` has type `PushHistory -> Prop`. A valid history must:

1. use nonempty commit ids,
2. chain adjacent steps correctly,
3. keep all steps on the same file, and
4. keep commit ids distinct across the whole linear history.

This means histories with repeated commit ids are rejected rather than treated as ambiguous ancestry.

9.5 Shared Prefix Logic

Definition 9.8 (commonPrefix)**.** `[commonPrefix]` returns the longest shared prefix of two lists.

Definition 9.9 (sharedCommitPath)**.** `[sharedCommitPath]` applies `commonPrefix` to the two histories' `commitPath` values.

Definition 9.10 (sharedStepCount)**.** `[sharedStepCount]` returns the number of shared patch steps, computed from the shared commit path length.

Definition 9.11 (CommonParentSplit)**.** `[CommonParentSplit]` packages the shared commit ids, the nearest common ancestor commit id, and the left and right residual histories.

Definition 9.12 (splitAtNearestCommonParent?)**.** `[splitAtNearestCommonParent?]` computes the shared commit path. If there is no shared commit id, it returns `none`. Otherwise it returns the nearest common ancestor plus the residual histories obtained by dropping the shared steps.

9.6 Theorems

Theorem 9.1 (commitPath_nil)**.** `[commitPath_nil]` states that the commit path of the empty history is `[]`.

Theorem 9.2 (commitPath_singleton)**.** `[commitPath_singleton]` states that the commit path of `[step]` is `[step.baseCommit, step.newCommit]`.

Theorem 9.3 (validHistory_distinctCommitIds)**.** `[validHistory_distinctCommitIds]` states that every valid history has a duplicate-free commit path.

Theorem 9.4 (empty shared path gives no split)**.** If the shared commit path is empty, then there is no common-parent split.

Theorem 9.5 (split_eq_some_of_sharedCommitPath)**.** `[split_eq_some_of_sharedCommitPath]` states the exact split returned when the shared commit path is nonempty.

Theorem 9.6 (validHistory_tail)**.** `[validHistory_tail]` states that the tail of a valid nonempty history is also valid.

Theorem 9.7 (validHistory_drop)**.** `[validHistory_drop]` states that dropping any number of steps from a valid history leaves a valid history.

Theorem 9.8 (validHistory_leftResidual)**.** `[validHistory_leftResidual]` states that the left residual returned by a valid split is valid.

Theorem 9.9 (validHistory_rightResidual)**.** `[validHistory_rightResidual]` states that the right residual returned by a valid split is valid.

9.7 Execution Pipeline

The history pipeline is:

1. represent a linear history as `PushHistory`,
2. check chaining, file consistency, and distinct commit ids with `validHistory`,
3. convert each history to a commit-id path with `commitPath`,
4. find the shared commit-id prefix with `sharedCommitPath`,
5. split at the nearest common ancestor with `splitAtNearestCommonParent?`,
6. pass the residual histories to `FullMerge.lean`.

10 FullMerge.lean

`FullMerge.lean` connects the previous layers into a merge pipeline. It replays histories, finds residual diffs, detects conflicts, constructs clean merge results, and defines multi-agent compatibility.

10.1 Imports and Namespace

This module builds on `Defns.lean`, `Semantics.lean`, `History.lean`, and `Conflict.lean`. It uses `Mathlib.Tactic` for the short compatibility proofs.

10.2 Merge Types

Definition 10.1 (mergeArtifactAgent and mergeArtifactTimestamp)**.** `[mergeArtifactAgent]` and `[mergeArtifactTimestamp]` are the canonical metadata used for clean merge artifacts. A merged diff is constructed by the merge pipeline rather than authored by one input agent, so the code makes that canonical metadata explicit instead of hard-coding anonymous literals.

Definition 10.2 (MergeM)**.** `[MergeM]` is an abbreviation for `Except String`. It marks merge pipeline computations that can fail with an error message.

Definition 10.3 (MergeResult)**.** `[MergeResult]` is an inductive type with constructors `MergeResult.clean` and `MergeResult.conflict`. `clean` carries a merged `Diff`. `conflict` carries a list of conflicting `Hunk` witnesses.

10.3 Replay

Definition 10.4 (replayFromCommon)**.** `[replayFromCommon]` has type `BaseFile -> PushHistory -> MergeM BaseFile`. It folds through a residual history and applies each step's diff to the current file with `applyDiff`.

Theorem 10.1 (replayFromCommon_nil)**.** `[replayFromCommon_nil]` states that replaying an empty history returns the original base file.

Theorem 10.2 (replayFromCommon_append)**.** `[replayFromCommon_append]` states that replaying `h1 ++ h2` is the same as replaying `h1` and then replaying `h2` from the intermediate file.

10.4 Residual Diffs and Merge Artifacts

Definition 10.5 (residualDiffs)**.** `[residualDiffs]` maps a `PushHistory` to the list of `Diff` values stored in its patch steps.

Definition 10.6 (conflictWitnesses)**.** `[conflictWitnesses]` takes left and right diff lists and returns the hunks from both sides that participate in a cross-side conflict according to `hunkConflictFlag`.

Definition 10.7 (dedupeIdenticalSortedHunks)**.** `[dedupeIdenticalSortedHunks]` removes adjacent duplicate hunks from a sorted hunk list when the duplicates are literally the same edit. This prevents clean merges of identical edits from producing an invalid overlapping merged diff.

Definition 10.8 (mergedDiffFrom)**.** `[mergedDiffFrom]` constructs a merged `Diff` from a common base file and two residual diff lists. It chooses a file name, concatenates all hunks from both sides, sorts them with `Hunk.compareByRange`, removes identical duplicates, renumbers their `seq` fields, sets `base\_line\_count` to `common.length`, and uses the canonical merge artifact metadata.

Theorem 10.3 (residualDiffs_nil)**.** `[residualDiffs_nil]` states that the residual diffs of the empty history are `[]`.

Theorem 10.4 (residualDiffs_cons)**.** `[residualDiffs_cons]` states that residual diffs of `step :: rest` are `step.diff :: residualDiffs rest`.

10.5 Three-Way Check

Definition 10.9 (threeWayCheck)**.** `[threeWayCheck]` has type `BaseFile -> PushHistory -> PushHistory -> MergeM (Option MergeResult)`. It replays the left and right residual histories from the common file. It then checks whether any residual left diff conflicts with any residual right diff. If a conflict exists, it returns `some (.conflict witnesses)`. Otherwise, it returns `some (.clean mergedDiff)`.

Theorem 10.5 (threeWayCheck_of_conflictFlag_true)**.** `[threeWayCheck_of_conflictFlag_true]` states the exact output of `threeWayCheck` when both residual histories replay successfully and the cross-history conflict flag is `true`.

Theorem 10.6 (threeWayCheck_of_conflictFlag_false)**.** `[threeWayCheck_of_conflictFlag_false]` states the exact output of `threeWayCheck` when both residual histories replay successfully and the cross-history conflict flag is `false`.

10.6 Full Merge

Definition 10.10 (reconstructAncestor)**.** `[reconstructAncestor]` has type `BaseFile -> PushHistory -> PushHistory -> MergeM BaseFile`. It replays the shared prefix of the left history from the initial file, using `sharedStepCount left right`.

Definition 10.11 (fullMerge)**.** `[fullMerge]` has type `BaseFile -> PushHistory -> PushHistory -> MergeM (Option MergeResult)`. It calls `splitAtNearestCommonParent?`. If there is no common parent, it returns `none`. If a split exists, it reconstructs the ancestor file and runs `threeWayCheck` on the residual histories.

10.7 N-Way Compatibility

Definition 10.12 (nWayCompatible.aux)**.** `[nWayCompatible.aux]` checks whether one diff is conflict-free against every diff in a list.

Theorem 10.7 (nWayCompatible_aux_nil)**.** `[nWayCompatible_aux_nil]` states that the auxiliary check against an empty list returns `true`.

Theorem 10.8 (nWayCompatible_aux_cons)**.** `[nWayCompatible_aux_cons]` states the recursive equation for checking one diff against `d2 :: rest`.

Definition 10.13 (nWayCompatible)**.** `[nWayCompatible]` has type `List Diff -> Bool`. It checks whether all pairs of diffs in a list are conflict-free.

Theorem 10.9 (nWayCompatible_nil)**.** `[nWayCompatible_nil]` states that the empty list is n-way compatible.

Theorem 10.10 (nWayCompatible_cons)**.** `[nWayCompatible_cons]` states the recursive equation for `nWayCompatible (d :: rest)`.

Theorem 10.11 (nWayCompatible_singleton)**.** `[nWayCompatible_singleton]` states that a singleton diff list is compatible.

Definition 10.14 (PairwiseCompatible)**.** `[PairwiseCompatible]` is the proposition-level version of `nWayCompatible`. It states that each diff is compatible with every later diff in the list.

Theorem 10.12 (PairwiseCompatible_nil)**.** `[PairwiseCompatible_nil]` states that the empty list is pairwise compatible.

Theorem 10.13 (PairwiseCompatible_cons)**.** `[PairwiseCompatible_cons]` states the recursive proposition-level form of pairwise compatibility.

Theorem 10.14 (nWayCompatible_eq_true_iff)**.** `[nWayCompatible_eq_true_iff]` states that `nWayCompatible ds = true` is equivalent to `PairwiseCompatible ds`.

Theorem 10.15 (nWayCompatible_sound)**.** `[nWayCompatible_sound]` states that if the boolean n-way compatibility check returns `true`, then `PairwiseCompatible` holds.

Theorem 10.16 (nWayCompatible_complete)**.** `[nWayCompatible_complete]` states that if `PairwiseCompatible` holds, then the boolean n-way compatibility check returns `true`.

10.8 Valid Merge

Definition 10.15 (ValidMerge)**.** `[ValidMerge]` has type `List Diff -> Prop`. It states that the list has at least two diffs and is `PairwiseCompatible`.

Theorem 10.17 (validMerge_iff)**.** `[validMerge_iff]` unfolds `ValidMerge`: it is equivalent to `2 <= diffs.length` and `PairwiseCompatible diffs`.

Theorem 10.18 (validMerge_iff_flag)**.** `[validMerge_iff_flag]` connects the proposition-level validity condition to the boolean check: `ValidMerge diffs` is equivalent to `2 <= diffs.length` and `nWayCompatible diffs = true`.

Theorem 10.19 (validMerge_of_flag)**.** `[validMerge_of_flag]` states that enough diffs plus a true compatibility flag imply `ValidMerge`.

Theorem 10.20 (flag_of_validMerge)**.** `[flag_of_validMerge]` states that a valid merge has `nWayCompatible diffs = true`.

Theorem 10.21 (validMerge_has_two_or_more)**.** `[validMerge_has_two_or_more]` states that any valid merge has at least two diffs.

10.9 Common Parent Theorems

Theorem 10.22 (split_ne_none_of_shared_nonempty)**.** `[split_ne_none_of_shared_nonempty]` states that if two histories have a nonempty shared commit path, then `splitAtNearestCommonParent?` is not `none`.

Theorem 10.23 (common_parent_exists_for_two)**.** `[common_parent_exists_for_two]` states that two valid histories with a nonempty shared commit path have a common-parent split.

11 Open Work

The current code checks structured diffs. It parses JSON, validates hunks, checks the base file, applies diffs, detects conflicts, models linear histories, and checks merges. The following work is still open.

11.1 Monadic Pipeline

Add a monadic version of the pipeline. It must run parsing, schema checks, base checks, conflict checks, and merge checks in one sequence. The current code exposes these steps as separate functions.

11.2 Invalid Input Flags

Add a result type for bad input from an agent. It must separate JSON parse errors, schema errors, and base-alignment errors. The current code returns `Except String`; callers still have to classify the failure.

11.3 Merge Conflict Flags

Add a result type for merge conflicts. The current code computes conflict witnesses with `conflictWitnesses` and returns them through `MergeResult.conflict`. Callers still have to present that result as a clear merge-conflict flag.

11.4 Agent Runner

There is no file that runs agents. A separate runner can call `parseDiff`, `Diff.checkSchema`, `Diff.checkAgainstBase`, `nWayCompatible`, and `fullMerge`. The core definitions do not need to change for that runner.

11.5 Commit Graphs

`PushHistory` is a linear list of `PatchStep`s. It does not model branches or merge commits. A graph model needs a commit table, parent commit ids, and a nearest-common-ancestor function.

11.6 Merged Diff Metadata

`Diff` stores one `agent` and one `timestamp`. A clean merge may combine edits from multiple agents. A later version can add a field that lists all contributing agents and timestamps.

11.7 Timestamp Parser

Lean stores `timestamp` as `String`. A later version can add a `Timestamp` type and reject strings that are not valid ISO 8601 date-times.

11.8 Sequence Numbers

`seq` is checked to be positive. It is not checked to be exactly `1, 2, ..., n`. A later schema check can add that rule.

11.9 File IO

The core code has no file-system dependency. A separate executable can read a file, collect diff JSON, validate it, and print either a merge result or conflict witnesses.